REVISION CHEAT SHEET:

STRINGS / C#

Strings and String Processing in C#

Interview-ready notes · examples · C# · flows

Topic Coverage

Strings are everywhere in full-stack .NET development.

You use strings for:

EXAMPLE / FLOW

```
Names
Email addresses
URLs
JSON
HTTP headers
Route values
Query parameters
Database text
Configuration
Logging
Error messages
Search terms
Currency codes
Identifiers
File paths
```

Although a string looks simple:

C#

```
string donorName = "John Smith";
```

professional string handling involves several important concepts:

- `string` and `System.String`;
- reference-type behavior;
- string immutability;
- string literals;
- escape sequences;
- verbatim strings;
- raw string literals;
- interpolated strings;
- interpolated raw strings;
- string concatenation;
- `StringBuilder`;
- indexing and `Length`;
- searching;
- comparison;
- case-sensitive vs case-insensitive comparison;
- ordinal vs culture-sensitive comparison;

- `Substring`;
- ranges and slices;
- `Replace`;
- `Remove`;
- `Insert`;
- trimming;
- null, empty, and whitespace strings;
- `Split`;
- `Join`;
- formatting numbers and dates;
- parsing strings into other types;
- Unicode and UTF-16;
- `char`, `Rune`, and Unicode characters;
- UTF-8 literals;
- string interning;
- `ReadOnlySpan<char>`;
- performance;
- security considerations;
- ASP.NET Core and Fundraising Platform examples.

The main concept is:

EXAMPLE / FLOW

```
Text
    →
string
    →
Search / Compare / Format / Parse / Transform
    →
Business Data
```

What Is a String?

Definition

A **string is an object that represents text**.

Example:

C#

```
string donorName =
    "John Smith";
```

In C#:

C#

```
string
```

is an alias for:

C#

```
System.String
```

These are equivalent:

```C#
string name =
    "John";
```

and:

```C#
System.String name =
    "John";
```

Microsoft recommends the C# keyword `string` for normal C# source code. A .NET string internally represents a sequence of `char` values.

string Is a Reference Type

From Topic 6, remember that:

```EXAMPLE / FLOW
string
    →
Reference Type
```

Example:

```C#
string first =
    "Annual Giving";

string second =
    first;
```

Both variables contain references.

However, strings behave differently from most mutable classes because strings are:

```EXAMPLE / FLOW
immutable
```

That distinction is extremely important.

What Does String Immutability Mean?

Definition

A string object cannot have its contents changed after that string object has been created.

Suppose:

```C#
string campaignName =
    "Annual";
```

Then:

```C#
campaignName +=
    " Giving";
```

looks as though the original string changed.

But it did not.

Conceptually:

```EXAMPLE / FLOW
"Annual"
    →
Create new string
"Annual Giving"
    →
campaignName now points to new string
```

The original `"Annual"` object remains unchanged. Microsoft explicitly states that string operations that appear to modify a string actually create and return a new string.

Immutability Example

```C#
string first =
    "Gift";

string second =
    first;

first +=
    " Processing";

Console.WriteLine(
    first);

Console.WriteLine(
    second);
```

Output:

```EXAMPLE / FLOW
Gift Processing
Gift
```

Why?

Initially:

```EXAMPLE / FLOW
first  ──┐
         ↓
       "Gift"
         ↑
second ──┘
```

Then:

C#

```
first +=
    " Processing";
```

creates another string:

EXAMPLE / FLOW

```
first
    →
"Gift Processing"

second
    →
"Gift"
```

`second` still references the original string.

Why Are Strings Immutable?

Immutability provides several benefits:

EXAMPLE / FLOW

```
Safer sharing

Predictable behavior

Thread-friendly read access

Hash-code stability

String interning support

Safer use as dictionary keys
```

For example, if strings were mutable and you used a string as a dictionary key, changing its contents could break lookup behavior.

Immutability makes a string value stable after creation.

Immutability Does Not Mean Variable Cannot Change

This is an important distinction.

C#

```
string donorName =
    "John";

donorName =
    "David";
```

is valid.

What changed?

EXAMPLE / FLOW

```
Variable reference
```

not:

EXAMPLE / FLOW

```
original string object
```

Flow:

EXAMPLE / FLOW

```
donorName
    →
"John"

Reassign
    →
donorName
    →
"David"
```

The variable can point to another string.

The string objects themselves remain immutable.

String Declaration

Examples:

C#

```
string donorName =
    "John Smith";
```

C#

```
string currencyCode =
    "USD";
```

C#

```
string campaignName =
    "Annual Giving 2026";
```

Nullable string:

C#

```
string? notes =
    null;
```

Empty string:

C#

```
string notes =
    "";
```

or:

C#

```
string notes =
    string.Empty;
```

null vs Empty vs Whitespace

STRINGS / C#

These are three different concepts.

null

```C#
string? notes =
    null;
```

Means:

EXAMPLE / FLOW

```
No string value exists.
```

Empty String

```C#
string notes =
    "";
```

Means:

EXAMPLE / FLOW

```
String exists
but contains zero characters.
```

Length:

```C#
notes.Length
```

is:

EXAMPLE / FLOW

```
0
```

Whitespace String

```C#
string notes =
    "   ";
```

The string contains characters, but they are whitespace.

Length:

EXAMPLE / FLOW

```
3
```

These states may require different business handling.

IsNullOrEmpty

Instead of:

```C#
if (value is null
    ||
    value.Length == 0)
{
}
```

you can write:

```C#
if (string.IsNullOrEmpty(
        value))
{
}
```

This returns true when the input is:

EXAMPLE / FLOW

```
null
or
""
```

IsNullOrWhiteSpace

More common for user-entered text:

```C#
if (string.IsNullOrWhiteSpace(
        donorName))
{
}
```

It treats:

EXAMPLE / FLOW

```
null

""

"   "

"\t"

"\r\n"
```

as missing/blank text.

Fundraising Validation Example

STRINGS / C#

```C#
if (string.IsNullOrWhiteSpace(
        currencyCode))
{
    throw new ArgumentException(
        "Currency code is required.",
        nameof(currencyCode));
}
```

Why `IsNullOrWhiteSpace` rather than only checking `null`?

Because this:

EXAMPLE / FLOW

```
"   "
```

should probably not be accepted as a currency code.

String Length

Use:

```C#
string value =
    "Gift";

int length =
    value.Length;
```

Result:

EXAMPLE / FLOW

```
4
```

However, an advanced Unicode rule is important:

> `string.Length` counts UTF-16 `char` code units, not necessarily the number of human-visible characters.

For many ordinary English strings these counts are the same, but emoji and characters outside the Basic Multilingual Plane can require two `char` values.

Accessing Individual Characters

Strings can be indexed.

```C#
string value =
    "Gift";

char first =
    value[0];
```

Result:

EXAMPLE / FLOW

```
'G'
```

Indexes are zero-based:

EXAMPLE / FLOW

```
G  i  f  t
0  1  2  3
```

String Index Is Read-Only

This is invalid:

C#

```
string value =
    "Gift";

// Invalid:
// value[0] = 'L';
```

Why?

Because strings are immutable.

If individual characters must be changed repeatedly, a different approach such as:

EXAMPLE / FLOW

```
StringBuilder

char[]

Span<char>
```

may be more appropriate depending on the scenario.

Basic String Literals

A normal string literal uses double quotes:

C#

```
string name =
    "Annual Giving";
```

Escape Sequences

Some characters require escape sequences.

Example:

C#

```
string message =
    "Line 1\nLine 2";
```

Common escape sequences:

Escape	Meaning
`\n`	New line
`\r`	Carriage return
`\t`	Tab
`\\`	Backslash
`\"`	Double quote
`\'`	Single quote
`\0`	Null character
`\uXXXX`	Unicode escape

New Line Example

C#
```
string receipt =
    "Gift received\nThank you";
```

Output:

EXAMPLE / FLOW
```
Gift received
Thank you
```

Quote Example

C#
```
string message =
    "The status is \"Processed\".";
```

Output:

EXAMPLE / FLOW
```
The status is "Processed".
```

Windows Path Example

Normal literal:

C#
```
string path =
    "C:\\Reports\\Gifts\\2026";
```

Because:

EXAMPLE / FLOW
```
\
```

has special meaning inside regular strings.

Verbatim String Literal

STRINGS / C#

A verbatim string begins with:

EXAMPLE / FLOW

```
@
```

Example:

C#

```
string path =
    @"C:\Reports\Gifts\2026";
```

Backslashes don't need to be escaped.

This is useful for:

EXAMPLE / FLOW

```
File paths

Regular expressions

Multi-line text
```

Quotes Inside Verbatim Strings

In verbatim strings, use double double-quotes:

C#

```
string message =
    @"Gift status is ""Processed"".";
```

Result:

EXAMPLE / FLOW

```
Gift status is "Processed".
```

Multi-Line Verbatim String

C#

```
string message =
    @"Gift received.
Thank you for your support.
Annual Giving Campaign";
```

Line breaks in a verbatim string are part of the resulting string.

Raw String Literals

Modern C# supports **raw string literals**.

They are especially useful for:

EXAMPLE / FLOW

STRINGS / C#

```
JSON

XML

SQL

HTML

Regex

Multi-line templates
```

Basic syntax:

C#

```
string message =
    """
    Thank you for your gift.
    Your receipt is attached.
    """;
```

Raw strings begin and end with at least three double quotes, can span multiple lines, and reduce the need for escape sequences.

Raw JSON Example

Without raw strings:

C#

```
string json =
    "{ \"giftId\": 1001, \"amount\": 1000 }";
```

With raw string:

C#

```
string json =
    """
    {
      "giftId": 1001,
      "amount": 1000
    }
    """;
```

Much easier to read.

Raw SQL Example

C#

```
string sql =
    """
    SELECT
        GiftId,
        ConstituentId,
        TotalAmount
    FROM fundraising.Gift
```

```
    WHERE TenantId = @TenantId;
    """;
```

STRINGS / C#

Important:

Using raw strings does **not** make SQL safe from injection.

Parameterization is still required.

Raw String Quotes

If your string itself contains:

```
EXAMPLE / FLOW
"""
```

you can use a longer delimiter.

For example:

```
C#
string value =
    """"
    Text containing """ inside it.
    """";
```

The delimiter count determines what quote sequences can appear inside.

String Interpolation

Definition

String interpolation lets you insert expressions inside text.

Syntax:

```
C#
$"...{expression}..."
```

Example:

```
C#
string donorName =
    "John";

decimal giftAmount =
    1000m;

string message =
    $"{donorName} donated {giftAmount}.";
```

Result:

```
EXAMPLE / FLOW
John donated 1000.
```

Microsoft recommends interpolation for readable inline formatting compared with manually numbered `String.Format` placeholders.

Fundraising Example

```C#
long giftId =
    1001;

decimal amount =
    1250m;

string message =
    $"Gift {giftId} received for {amount}.";
```

Expressions Inside Interpolation

You can use expressions:

```C#
decimal giftAmount =
    1000m;

decimal fee =
    25m;

string message =
    $"Net amount: {giftAmount - fee}";
```

Result:

```EXAMPLE / FLOW
Net amount: 975
```

Interpolated Formatting

Interpolated expressions support format specifiers.

Example:

```C#
decimal amount =
    1234.5m;

string message =
    $"Gift amount: {amount:C2}";
```

With an en-US current culture, that may display:

```EXAMPLE / FLOW
Gift amount: $1,234.50
```

Numeric formatting is culture-sensitive unless a specific culture/provider is supplied. Standard numeric format specifiers include `C`, `N`, `F`, `P`, `D`, `X`, and others.

Useful Numeric Format Specifiers

STRINGS / C#

Format	Meaning
`C`	Currency
`N`	Number
`F`	Fixed point
`P`	Percent
`D`	Decimal integer
`X`	Hexadecimal

Examples:

```C#
decimal amount =
    1234.567m;

Console.WriteLine(
    $"{amount:C2}");

Console.WriteLine(
    $"{amount:N2}");

Console.WriteLine(
    $"{amount:F2}");
```

Percentage Formatting

```C#
decimal completion =
    0.75m;

string message =
    $"{completion:P0}";
```

Typical output:

EXAMPLE / FLOW

```
75%
```

Date Formatting

```C#
DateTime createdAtUtc =
    DateTime.UtcNow;

string message =
    $"{createdAtUtc:yyyy-MM-dd}";
```

Standard and custom date/time format strings work with `DateTime`, `DateTimeOffset`, `DateOnly`, and `TimeOnly`; some standard formats such as `"O"` provide culture-independent represen

STRINGS / C#

round-tripping.

Round-Trip Date Format

For technical persistence/logging:

C#

```
string timestamp =
    DateTime.UtcNow.ToString(
        "O");
```

`"O"` means:

EXAMPLE / FLOW

```
Round-trip format
```

This is generally preferable to user-facing formats when machine-readable precision is needed.

Alignment in Interpolated Strings

Example:

C#

```
string message =
    $"{giftId,-10}{amount,12:C2}";
```

The comma specifies alignment width.

This can help with:

EXAMPLE / FLOW

```
Console reports

Fixed-width text

Simple exports
```

Interpolated Verbatim Strings

You may encounter:

C#

```
string path =
    $@"C:\Users\{userName}\Reports";
```

This combines:

EXAMPLE / FLOW

```
$
    →
interpolation

@
```

```
    →
verbatim string
```

STRINGS / C#

Interpolated Raw Strings

Modern code can often be cleaner using:

```C#
string path =
    $"""C:\Users\{userName}\Reports""";
```

Microsoft's current guidance notes that interpolated raw strings are especially convenient for paths and text that contains backslashes or quotation marks.

Interpolated Raw JSON Example

```C#
long giftId =
    1001;

decimal amount =
    1000m;

string json =
    $$"""
    {
      "giftId": {{giftId}},
      "amount": {{amount}}
    }
    """;
```

Why two `$` characters?

The number of `$` characters determines how many braces are required to identify interpolation.

This is useful when the content itself contains many `{}` characters, such as JSON.

String Concatenation

Strings can be concatenated using:

EXAMPLE / FLOW

```
+
```

Example:

```C#
string firstName =
    "John";

string lastName =
    "Smith";

string fullName =
    firstName
    +
    " "
```

```
    +
    lastName;
```

STRINGS / C#

Result:

EXAMPLE / FLOW

```
John Smith
```

Concatenation Creates Strings

Because strings are immutable:

C#

```
string result =
    first + second;
```

must produce a resulting string.

For a small number of concatenations, this is perfectly normal.

Do not automatically replace every `+` with `StringBuilder`.

String.Concat

You can also use:

C#

```
string result =
    string.Concat(
        firstName,
        " ",
        lastName);
```

Microsoft's `String` API provides `Concat` for combining values and `Join` for combining a sequence with separators.

String.Join

Suppose:

C#

```
string[] funds =
{
    "General",
    "Research",
    "Education"
};
```

Use:

C#

```
string result =
    string.Join(
        ", ",
        funds);
```

Result:

EXAMPLE / FLOW

```
General, Research, Education
```

This is usually much cleaner than manually building separators in a loop.

Bad Manual Joining

Avoid:

C#

```
string result =
    "";

foreach (string fund in funds)
{
    result +=
        fund + ", ";
}
```

Problems:

EXAMPLE / FLOW

```
Extra separator

Repeated string creation

More complicated logic
```

Use:

C#

```
string.Join(...)
```

when it matches the requirement.

StringBuilder

Definition

`StringBuilder` is a mutable text buffer from:

C#

```
System.Text
```

Unlike `string`, its internal contents can be modified repeatedly.

Example:

C#

```
using System.Text;

var builder =
    new StringBuilder();

builder.Append(
```

```
    "Gift");

builder.Append(
    " ");

builder.Append(
    "Processed");

string result =
    builder.ToString();
```

STRINGS / C#

Result:

```
EXAMPLE / FLOW
Gift Processed
```

Microsoft recommends `StringBuilder` when code performs many repeated string modifications, particularly inside loops where immutable string operations can create many intermediate strings.

Why StringBuilder Exists

Consider:

```
C#
string report =
    "";

for (int i = 0;
     i < 10_000;
     i++)
{
    report +=
        $"Gift {i}\n";
}
```

Conceptually:

```
EXAMPLE / FLOW
String 1
    →
new String 2
    →
new String 3
    →
new String 4
    →
...
```

Repeated concatenation can produce many temporary string objects.

StringBuilder Flow

```
EXAMPLE / FLOW
StringBuilder created
    →
Append text
    →
```

```
Append more text
    →
Append more text
    →
Internal buffer grows
    →
ToString()
    →
Final immutable string
```

STRINGS / C#

StringBuilder Example — Gift Report

C#

```
using System.Text;

var builder =
    new StringBuilder();

foreach (Gift gift
         in gifts)
{
    builder.Append(
        "GiftId: ");

    builder.Append(
        gift.GiftId);

    builder.Append(
        ", Amount: ");

    builder.AppendLine(
        gift.TotalAmount
            .ToString("C2"));
}

string report =
    builder.ToString();
```

Useful StringBuilder Methods

Common methods:

EXAMPLE / FLOW

```
Append

AppendLine

AppendFormat

Insert

Remove

Replace

Clear

ToString
```

Example:


```C#
builder.AppendLine(
    "Gift Summary");
```

StringBuilder Capacity

You can optionally provide an initial capacity:

```C#
var builder =
    new StringBuilder(
        capacity: 1024);
```

This can reduce internal buffer resizing when you have a reasonable estimate of final size.

Do not guess huge capacities without reason.

When to Use StringBuilder

Good situations:

EXAMPLE / FLOW

```
Large loop

Many incremental appends

Building large text document

Repeated character modification

Unknown but potentially large output
```

When Not to Use StringBuilder

For:

```C#
string fullName =
    firstName
    +
    " "
    +
    lastName;
```

a `StringBuilder` would usually add unnecessary complexity.

Likewise:

```C#
string message =
    $"Gift {giftId} processed.";
```

is already simple and readable.

Modern C# and .NET optimize many common string-construction patterns, so use `StringBuilder` where repeated mutable construction actually makes sense rather than mechanically everyw

STRINGS / C#

String Search Operations

Common search methods include:

EXAMPLE / FLOW

```
Contains

StartsWith

EndsWith

IndexOf

LastIndexOf
```

Contains

C#

```
string campaignName =
    "Annual Giving 2026";

bool contains =
    campaignName.Contains(
        "Giving",
        StringComparison.Ordinal);
```

Result:

EXAMPLE / FLOW

```
true
```

StartsWith

C#

```
bool startsWithAnnual =
    campaignName.StartsWith(
        "Annual",
        StringComparison.Ordinal);
```

EndsWith

C#

```
bool currentYearCampaign =
    campaignName.EndsWith(
        "2026",
        StringComparison.Ordinal);
```

IndexOf


```C#
int index =
    campaignName.IndexOf(
        "Giving",
        StringComparison.Ordinal);
```

If found:

```EXAMPLE / FLOW
index >= 0
```

If not found:

```EXAMPLE / FLOW
-1
```

Prefer Contains for Simple Presence Checks

Instead of:

```C#
if (campaignName.IndexOf(
        "Annual",
        StringComparison.Ordinal)
    >= 0)
{
}
```

prefer:

```C#
if (campaignName.Contains(
        "Annual",
        StringComparison.Ordinal))
{
}
```

when you only need a Boolean result.

Current Microsoft guidance similarly recommends straightforward `Contains`, `StartsWith`, or `IndexOf` calls for simple searches before moving to more advanced span-based techniques.

String Comparison Is More Complicated Than It Looks

Consider:

```C#
string first =
    "USD";

string second =
    "usd";
```

Should these be considered equal?

Maybe:

EXAMPLE / FLOW

```
Yes
```

for a currency-code comparison.

But perhaps:

EXAMPLE / FLOW

```
No
```

for some case-sensitive identifier.

Comparison behavior must match the meaning of the data.

== with Strings

Unlike ordinary reference types, `string` overloads equality operators to compare string values.

Example:

C#

```
string first =
    "Gift";

string second =
    "Gift";

bool equal =
    first == second;
```

Result:

EXAMPLE / FLOW

```
true
```

even if the two variables happen to refer to different string objects.

string.Equals

Example:

C#

```
bool equal =
    string.Equals(
        first,
        second,
        StringComparison.Ordinal);
```

This is often preferable when comparison rules need to be explicit.

StringComparison

Important options include:

EXAMPLE / FLOW

```
Ordinal

OrdinalIgnoreCase

CurrentCulture

CurrentCultureIgnoreCase

InvariantCulture

InvariantCultureIgnoreCase
```

The choice matters for correctness and sometimes security.

Microsoft recommends explicitly specifying comparison behavior for string operations rather than relying on ambiguous defaults.

Ordinal Comparison

C#

```
string.Equals(
    first,
    second,
    StringComparison.Ordinal);
```

means:

EXAMPLE / FLOW

```
Compare Unicode values
without linguistic culture rules.
```

Use it for many machine-oriented identifiers.

Examples:

EXAMPLE / FLOW

```
Protocol tokens

Internal codes

Technical identifiers

JSON property names when application rules require exact case

Database/code keys
```

OrdinalIgnoreCase

Example:

C#

```
string.Equals(
    currencyCode,
    "USD",
    StringComparison.OrdinalIgnoreCase);
```

Then:

EXAMPLE / FLOW

```
USD
usd
Usd
```

can compare equal.

Microsoft recommends `Ordinal` or `OrdinalIgnoreCase` as a safe default for culture-independent, non-linguistic string matching.

Fundraising Currency Example

C#

```
if (string.Equals(
        currencyCode,
        "USD",
        StringComparison.OrdinalIgnoreCase))
{
    ProcessUsdGift();
}
```

This is preferable to:

C#

```
if (currencyCode.ToUpper()
    == "USD")
{
}
```

because you do not need to create another normalized string just to compare case-insensitively.

CurrentCulture Comparison

Use culture-sensitive comparison when text is genuinely linguistic and intended for users.

Example:

EXAMPLE / FLOW

```
Names

Words

User-facing sorting
```

Culture affects:

EXAMPLE / FLOW

```
Letter ordering

Case rules

Linguistic equivalence
```

Microsoft recommends culture-aware operations for text intended for human linguistic display and ordinal comparisons for culture-independent identifiers.

Do Not Compare Technical Identifiers Using Current Culture

STRINGS / C#

Bad concept:

```C#
if (technicalToken.Equals(
        expectedToken,
        StringComparison.CurrentCultureIgnoreCase))
{
}
```

If the value is a technical token, protocol value, security setting, or internal identifier, culture rules are usually inappropriate.

Prefer:

```C#
StringComparison.Ordinal
```

or:

```C#
StringComparison.OrdinalIgnoreCase
```

depending on the specification.

This avoids subtle globalization and security problems.

Case Conversion Is Not the Best Comparison Strategy

Avoid:

```C#
if (first.ToLower()
    ==
    second.ToLower())
{
}
```

Problems:

EXAMPLE / FLOW

```
Extra allocations

Culture behavior

Intent less explicit
```

Prefer:

```C#
if (string.Equals(
        first,
        second,
        StringComparison.OrdinalIgnoreCase))
```

```
{
}
```

STRINGS / C#

StringComparer

For collections such as dictionaries and sets, use an appropriate comparer.

Example:

C#

```
var currencyRates =
    new Dictionary<string, decimal>(
        StringComparer.OrdinalIgnoreCase);
```

Then:

EXAMPLE / FLOW

```
USD
usd
Usd
```

are treated consistently as the same dictionary key according to the comparer.

This is much better than manually lowercasing keys everywhere.

Substring

Use:

C#

```
string value =
    "Annual Giving";

string part =
    value.Substring(
        0,
        6);
```

Result:

EXAMPLE / FLOW

```
Annual
```

Remember:

EXAMPLE / FLOW

```
Substring creates a new string.
```

Microsoft lists `Substring` as the standard operation for extracting part of a string, and because strings are immutable the result is a new string.

Range Syntax

Modern C# provides range syntax.

```C#
string code =
    "GIFT-2026";
```

First four characters:

```C#
string prefix =
    code[..4];
```

Result:

```
EXAMPLE / FLOW
GIFT
```

From position 5 onward:

```C#
string year =
    code[5..];
```

Result:

```
EXAMPLE / FLOW
2026
```

Index from End

C# also supports:

```
EXAMPLE / FLOW
^
```

Example:

```C#
char last =
    code[^1];
```

Meaning:

```
EXAMPLE / FLOW
last character
```

For:

```
EXAMPLE / FLOW
GIFT-2026
```

result:

```
EXAMPLE / FLOW
'6'
```

String.Replace


```C#
string value =
    "Gift Pending";

string updated =
    value.Replace(
        "Pending",
        "Processed");
```

Result:

```EXAMPLE / FLOW
Gift Processed
```

Original:

```EXAMPLE / FLOW
Gift Pending
```

still exists unchanged because `Replace` returns a new string.

Common Immutability Mistake

Wrong:

```C#
string status =
    "Pending";

status.Replace(
    "Pending",
    "Processed");

Console.WriteLine(
    status);
```

Output remains:

```EXAMPLE / FLOW
Pending
```

Correct:

```C#
status =
    status.Replace(
        "Pending",
        "Processed");
```

Trim

```C#
string input =
    "   John Smith   ";

string trimmed =
    input.Trim();
```

Result:

```EXAMPLE / FLOW
John Smith
```

TrimStart

```C#
string value =
    "   Gift";

string result =
    value.TrimStart();
```

Result:

```EXAMPLE / FLOW
Gift
```

TrimEnd

```C#
string value =
    "Gift   ";

string result =
    value.TrimEnd();
```

Result:

```EXAMPLE / FLOW
Gift
```

`Trim`, `TrimStart`, and `TrimEnd` remove whitespace or specified characters from the appropriate boundaries and return new strings.

Trim Does Not Remove Interior Spaces

```C#
string value =
    "John   Smith";
```

```
string result =
    value.Trim();
```

STRINGS / C#

The interior spaces remain:

EXAMPLE / FLOW

```
John   Smith
```

`Trim` only affects string boundaries.

Remove

C#

```
string value =
    "Gift-TEST";

string result =
    value.Remove(
        4);
```

Result:

EXAMPLE / FLOW

```
Gift
```

Or remove a specific range:

C#

```
string result =
    value.Remove(
        4,
        5);
```

Insert

C#

```
string value =
    "Gift2026";

string result =
    value.Insert(
        4,
        "-");
```

Result:

EXAMPLE / FLOW

```
Gift-2026
```

Again, a new string is produced.

Split

Suppose:

```C#
string value =
    "USD,EUR,GBP";
```

Use:

```C#
string[] currencies =
    value.Split(
        ',');
```

Result conceptually:

EXAMPLE / FLOW

```
[0] USD
[1] EUR
[2] GBP
```

`String.Split` supports delimiter choices, limits, removal of empty entries, and trimming options.

Split with Options

Example:

```C#
string value =
    "USD, EUR, ,GBP";

string[] currencies =
    value.Split(
        ',',
        StringSplitOptions.TrimEntries
        |
        StringSplitOptions.RemoveEmptyEntries);
```

Result:

EXAMPLE / FLOW

```
USD
EUR
GBP
```

This is useful when processing comma-separated input.

Do Not Use Split for Every Parsing Problem

For small input:

```C#
"key=value"
```

`Split` is perfectly fine.

For extremely large parsing loops:

EXAMPLE / FLOW

```
Millions of lines

Large payloads

Hot performance paths
```

creating many substring objects can matter.

Later we will see:

EXAMPLE / FLOW

```
ReadOnlySpan<char>
```

as an allocation-conscious option.

Join

Opposite of splitting:

C#

```
string[] values =
{
    "USD",
    "EUR",
    "GBP"
};

string result =
    string.Join(
        ",",
        values);
```

Result:

EXAMPLE / FLOW

```
USD,EUR,GBP
```

Parsing Strings Into Numbers

Web applications frequently receive text that must become strongly typed data.

Example:

C#

```
string input =
    "1000.50";
```

Convert safely:

C#

```
if (decimal.TryParse(
        input,
        out decimal amount))
{
```

```
    // amount can be used.
}
```

STRINGS / C#

This was introduced earlier, but strings are where parsing begins.

Parsing Should Consider Culture

Suppose:

EXAMPLE / FLOW

```
1,234.50
```

Depending on culture:

EXAMPLE / FLOW

```
,
```

and:

EXAMPLE / FLOW

```
.
```

can represent different grouping/decimal meanings.

For machine-defined formats, provide an explicit culture and number style when necessary.

Example:

C#

```
using System.Globalization;

bool success =
    decimal.TryParse(
        input,
        NumberStyles.Number,
        CultureInfo.InvariantCulture,
        out decimal amount);
```

User Input vs Machine Input

These require different approaches.

User-facing input:

EXAMPLE / FLOW

```
May follow user's culture.
```

Machine protocol:

EXAMPLE / FLOW

```
Should follow a defined fixed format.
```

Do not use current culture accidentally for machine data.

Formatting for Humans

Example:

```C#
decimal amount =
    1234.50m;

string display =
    amount.ToString(
        "C2");
```

This uses current culture.

Good for:

EXAMPLE / FLOW

UI display

Formatting for Machines

For a culture-independent decimal representation:

```C#
string persisted =
    amount.ToString(
        CultureInfo.InvariantCulture);
```

Microsoft recommends culture-sensitive formatting for user display and invariant formatting when persisting non-string data in a stable machine-readable textual form.

Strings and JSON

In an ASP.NET Core Web API, you normally should **not manually build JSON** like:

```C#
string json =
    "{ \"giftId\": "
    + gift.GiftId
    + " }";
```

Instead use serialization:

```C#
JsonSerializer.Serialize(
    gift);
```

Why?

A serializer correctly handles:

EXAMPLE / FLOW

Escaping

Quotes

```
Nulls
Nested objects

Numbers

Dates

Unicode
```

STRINGS / C#

Manual string construction is error-prone.

Raw strings are useful for:

EXAMPLE / FLOW

```
Test fixtures

Static JSON templates

Examples
```

but not as a replacement for proper serialization.

Strings and SQL

Never write:

C#

```
string sql =
    "SELECT * FROM Gift "
    +
    "WHERE ReferenceNumber = '"
    +
    referenceNumber
    +
    "'";
```

This can create:

EXAMPLE / FLOW

```
SQL Injection
```

Use parameters:

C#

```
command.CommandText =
    """
    SELECT GiftId, TotalAmount
    FROM fundraising.Gift
    WHERE ReferenceNumber = @ReferenceNumber;
    """;
```

Then add the parameter through the database API.

String syntax and SQL safety are separate concerns.

Strings and URLs

STRINGS / C#

Avoid manually concatenating complex URLs when structured APIs are available.

For simple values:

```C#
string endpoint =
    $"/api/gifts/{giftId}";
```

is fine.

But user-controlled query values should be correctly encoded rather than concatenated without escaping.

StringBuilder and Logging

Avoid eagerly creating expensive log strings when structured logging is available.

Instead of:

```C#
_logger.LogInformation(
    "Gift "
    +
    giftId
    +
    " processed for "
    +
    amount);
```

prefer structured logging:

```C#
_logger.LogInformation(
    "Gift {GiftId} processed for {Amount}",
    giftId,
    amount);
```

This preserves structured values for log providers and is more useful for searching/telemetry.

String formatting and structured logging solve different problems.

Unicode and UTF-16

.NET `string` uses UTF-16 code units.

A:

```C#
char
```

represents one 16-bit UTF-16 code unit.

For ordinary characters such as:

EXAMPLE / FLOW

```
A
B
```

```
1
€
```

STRINGS / C#

one `char` may represent the full character.

But not always.

Emoji Example

Consider:

```C#
string emoji =
    "🐂";
```

A developer might expect:

```C#
emoji.Length
```

to be:

```EXAMPLE / FLOW
1
```

But UTF-16 represents that Unicode scalar value using a surrogate pair, so the string length is:

```EXAMPLE / FLOW
2
```

Microsoft's Unicode documentation specifically demonstrates that one displayed symbol or Unicode scalar value can require two `char` values.

char Is Not Always a Whole Human Character

This is an important advanced rule.

Do not assume:

```EXAMPLE / FLOW
1 char
=
1 Unicode character
=
1 visible symbol
```

Unicode is more complicated.

Possible concepts include:

```EXAMPLE / FLOW
UTF-16 code unit

Unicode scalar value

Grapheme cluster
```

Rune

.NET provides:

```
C#
System.Text.Rune
```

to represent a Unicode scalar value.

Example concept:

```
C#
foreach (Rune rune
         in text.EnumerateRunes())
{
}
```

This is safer than character-by-character `char` processing when you truly need Unicode scalar values.

When You Need Rune

Most normal business code does not need to manually use `Rune`.

Use it when implementing:

```
EXAMPLE / FLOW
Unicode-aware parsers

Character classification

Text algorithms

Normalization-like operations

Low-level international text processing
```

For normal display/search operations, framework string APIs usually handle Unicode more appropriately than manually processing each `char`.

UTF-8 vs UTF-16

Internally, .NET strings use:

```
EXAMPLE / FLOW
UTF-16
```

But many web protocols and file formats commonly use:

```
EXAMPLE / FLOW
UTF-8
```

Therefore applications frequently convert between:

EXAMPLE / FLOW

```
string
    ↔
UTF-8 bytes
```

Encoding UTF-8

C#

```
using System.Text;

string text =
    "Annual Giving";

byte[] bytes =
    Encoding.UTF8.GetBytes(
        text);
```

Decode:

C#

```
string restored =
    Encoding.UTF8.GetString(
        bytes);
```

Full-Stack Relevance

When your ASP.NET Core API sends JSON:

EXAMPLE / FLOW

```
C# string
    →
serializer
    →
text encoding
    →
HTTP bytes
```

When request arrives:

EXAMPLE / FLOW

```
HTTP bytes
    →
decoder
    →
JSON parser
    →
C# objects / strings
```

Understanding that strings and byte encodings are different is important.

UTF-8 String Literal

Modern C# supports:

```C#
ReadOnlySpan<byte> token =
    "AUTH "u8;
```

The `u8` suffix creates UTF-8 literal data represented as `ReadOnlySpan<byte>`, not a normal `string`. UTF-8 literals cannot currently be combined directly with interpolation and are not compile-time `const` values.

Why UTF-8 Literals Exist

They can be useful in:

```EXAMPLE / FLOW
Protocols

High-performance parsing

Native APIs

Byte-oriented processing
```

where you want UTF-8 bytes directly without first creating a UTF-16 string and encoding it.

For normal application text:

```C#
string
```

remains the normal choice.

ReadOnlySpan

This is an advanced performance concept.

Suppose:

```C#
string input =
    "key1=alpha;key2=beta";
```

You could extract substrings using:

```C#
Substring(...)
```

which creates new strings.

A span can instead create a lightweight view:

```C#
ReadOnlySpan<char> span =
    input.AsSpan();
```

Then slices:

```csharp
ReadOnlySpan<char> prefix =
    span[..4];
```

represent windows over existing characters rather than creating new string copies.

Microsoft recommends span-based parsing/search primarily for hot paths or large inputs where substring/split allocations have been measured as significant.

Span Example

```csharp
ReadOnlySpan<char> input =
    "GIFT-2026".AsSpan();

ReadOnlySpan<char> prefix =
    input[..4];

ReadOnlySpan<char> year =
    input[5..];
```

Conceptually:

```
Original Character Memory
G I F T - 2 0 2 6
└─────┘   └─────┘
 prefix      year
```

No independent prefix/year string has to be created merely for these views.

When Not to Use Span

Do not rewrite normal readable code:

```csharp
string year =
    code[5..];
```

into complicated span logic unless allocation/performance actually matters.

For most ASP.NET Core business logic:

```
Clarity first.
```

Use spans for:

```
Parser libraries

Protocol processing

High-volume text processing

Measured hot paths
```

String Interning

The CLR maintains an **intern pool** that can hold shared references for equal string values.

Example concept:

```C#
string first =
    "Gift";

string second =
    "Gift";
```

Equivalent string literals can share storage/reference identity under runtime/compiler rules.

You can also explicitly call:

```C#
string interned =
    string.Intern(
        value);
```

The intern pool returns a shared reference for equal interned values.

Do Not Depend on Interning for Equality

Never write business logic that assumes:

```C#
ReferenceEquals(
    first,
    second)
```

is the correct way to compare string contents.

Use:

```C#
string.Equals(...)
```

or:

```C#
==
```

with the appropriate comparison rules.

Interning is an implementation/runtime optimization concept, not your normal equality mechanism.

Manual Interning Can Increase Memory Lifetime

Interned strings can remain referenced by the intern pool for a very long time.

Microsoft notes that manually interning arbitrary strings can increase memory retention because interned values are unlikely to be released until the runtime terminates.

Do not call:

```
string.Intern(...)
```

on every user-provided or database string as an attempted optimization.

That can be much worse.

Common String Methods

A useful professional set to know:

EXAMPLE / FLOW

```
Length

Contains

StartsWith

EndsWith

IndexOf

LastIndexOf

Substring

Replace

Remove

Insert

Trim

TrimStart

TrimEnd

Split

ToUpper

ToLower

ToUpperInvariant

ToLowerInvariant

Equals

Compare

Concat

Join

IsNullOrEmpty

IsNullOrWhiteSpace
```

You do not need to memorize every overload.

Understand which operation each family provides.

Fundraising Example — Normalize Currency Input

Suppose user enters:

EXAMPLE / FLOW

```
" usd "
```

You might do:

C#

```
string normalizedCurrency =
    currencyCode.Trim()
                .ToUpperInvariant();
```

Result:

EXAMPLE / FLOW

```
USD
```

But for comparison alone:

C#

```
string.Equals(
    currencyCode.Trim(),
    "USD",
    StringComparison.OrdinalIgnoreCase)
```

is generally better than uppercasing solely for equality.

Normalization and comparison are different operations.

Fundraising Example — Reference Number Validation

C#

```
if (string.IsNullOrWhiteSpace(
        referenceNumber))
{
    throw new ArgumentException(
        "Reference number is required.");
}

string normalized =
    referenceNumber.Trim();
```

Then:

C#

```
if (!normalized.StartsWith(
        "GFT-",
        StringComparison.Ordinal))
{
    throw new ArgumentException(
```

```
        "Reference number is invalid.");
}
```


Fundraising Example — Parse Reference Number

Suppose:

EXAMPLE / FLOW

```
GFT-2026-10015
```

You could split:

C#

```
string[] parts =
    referenceNumber.Split(
        '-');

if (parts.Length == 3)
{
    string prefix =
        parts[0];

    string year =
        parts[1];

    string id =
        parts[2];
}
```

For occasional business parsing, this is perfectly reasonable.

For millions of records, allocation-conscious span parsing might become useful.

Fundraising Example — Donor Display Name

C#

```
string displayName =
    string.IsNullOrWhiteSpace(
        preferredName)

        ? legalName

        : preferredName.Trim();
```

This distinguishes meaningful user input from blank text.

Fundraising Example — Gift Receipt

C#

```
string receipt =
    $"""
    Gift Receipt

    Gift ID: {gift.GiftId}
    Donor: {donorName}
    Amount: {gift.TotalAmount:C2}
    Date: {gift.GiftDate:yyyy-MM-dd}
```

```
    Campaign: {campaignName}
    """;
```

STRINGS / C#

This is a good use of:

EXAMPLE / FLOW

```
Raw String
+
Interpolation
+
Formatting
```

Fundraising Example — Large Export

C#

```
var builder =
    new StringBuilder();

builder.AppendLine(
    "GiftId,ConstituentId,Amount");

foreach (Gift gift
         in gifts)
{
    builder.Append(
        gift.GiftId);

    builder.Append(',');

    builder.Append(
        gift.ConstituentId);

    builder.Append(',');

    builder.AppendLine(
        gift.TotalAmount
            .ToString(
                CultureInfo.InvariantCulture));
}

string csv =
    builder.ToString();
```

For production CSV, use a proper CSV writer if data can contain:

EXAMPLE / FLOW

```
commas

quotes

new lines
```

because correct CSV escaping is more complicated than simple concatenation.

Common Mistake — Forgetting Immutability

Wrong:

```C#
string value =
    " Pending ";

value.Trim();

Console.WriteLine(
    value);
```

Still:

EXAMPLE / FLOW

```
" Pending "
```

Correct:

```C#
value =
    value.Trim();
```

Common Mistake — Concatenating in Large Loop

Potentially inefficient:

```C#
string result =
    "";

foreach (Gift gift
         in gifts)
{
    result +=
        gift.GiftId
        +
        ",";
}
```

If this executes many thousands of times, repeated string creation can become expensive.

Consider:

```C#
StringBuilder
```

or:

```C#
string.Join
```

depending on the requirement.

Common Mistake — Using StringBuilder for Tiny Expressions

Overcomplicated:

```C#
var builder =
    new StringBuilder();

builder.Append(
    firstName);

builder.Append(
    " ");

builder.Append(
    lastName);

string fullName =
    builder.ToString();
```

Simpler:

```C#
string fullName =
    $"{firstName} {lastName}";
```

Common Mistake — Comparing Case by ToLower

Avoid:

```C#
if (currencyCode.ToLower()
    ==
    "usd")
{
}
```

Better:

```C#
if (string.Equals(
        currencyCode,
        "USD",
        StringComparison.OrdinalIgnoreCase))
{
}
```

This expresses intent directly and avoids unnecessary normalization allocation. Microsoft recommends explicit comparison rules.

Common Mistake — Using CurrentCulture for Security Tokens

Avoid linguistic comparison for:

```EXAMPLE / FLOW
Authorization scheme

Protocol token
```

```
Internal key
*
Security-sensitive identifier
```

STRINGS / C#

Use the comparison required by the protocol, usually ordinal-based.

Culture-dependent comparisons can cause unexpected and, in some cases, security-sensitive behavior.

Common Mistake — Confusing Empty and Null

This:

C#

```
name == ""
```

does not detect:

EXAMPLE / FLOW

```
null
```

Use:

C#

```
string.IsNullOrEmpty(
    name)
```

or:

C#

```
string.IsNullOrWhiteSpace(
    name)
```

depending on the requirement.

Common Mistake — Checking Only Length

Risky:

C#

```
if (name.Length == 0)
{
}
```

If `name` is null:

EXAMPLE / FLOW

```
NullReferenceException
```

Use a null-aware check when the variable can be null.

Common Mistake — Assuming Length Means User Characters

For Unicode text:

```C#
text.Length
```

counts UTF-16 code units.

It may not equal:

```EXAMPLE / FLOW
number of displayed symbols
```

Do not build Unicode-sensitive validation rules based only on `char` count without understanding the requirement.

Common Mistake — Truncating Unicode Arbitrarily

Suppose:

```C#
string text =
    "🐂";
```

A substring cutting between the UTF-16 surrogate pair can produce invalid/ill-formed UTF-16 data. Microsoft specifically notes that substring operations can split surrogate pairs if code slices blindly at arbitrary `char` boundaries.

For ordinary ASCII identifiers this isn't a problem.

For arbitrary user-visible Unicode, be more careful.

Common Mistake — Manual JSON Escaping

Avoid:

```C#
string json =
    "{\"name\":\""
    +
    userInput
    +
    "\"}";
```

If `userInput` contains:

```EXAMPLE / FLOW
quotes
backslashes
control characters
```

the generated JSON may be invalid or unsafe.

Use:

```C#
JsonSerializer
```

instead.

Common Mistake — SQL Concatenation

Never:

```C#
string query =
    "SELECT * FROM Gift WHERE Notes = '"
    +
    userInput
    +
    "'";
```

Use parameterized queries.

This is not merely a performance or style concern.

It is a security requirement.

Common Mistake — String IDs Everywhere

Avoid turning strongly typed data into strings too early.

Bad domain design:

```C#
string giftId =
    "1001";

string amount =
    "1000.00";

string refunded =
    "false";
```

Prefer:

```C#
long giftId =
    1001;

decimal amount =
    1000m;

bool refunded =
    false;
```

Strings are excellent for text.

They should not replace proper domain types.

Common Mistake — Formatting Before Calculating

Bad:

```C#
string amount =
    "$1,000.00";
```

then trying to perform arithmetic on it.

Better:

```C#
decimal amount =
    1000m;
```

Perform calculations with:

```
EXAMPLE / FLOW
decimal
```

Then format for display:

```C#
string display =
    $"{amount:C2}";
```

Common Mistake — Assuming ToString Is Persistence Format

Example:

```C#
string date =
    DateTime.Now.ToString();
```

This can depend on:

```
EXAMPLE / FLOW
Current culture
Machine configuration
```

For machine persistence, use an explicitly defined format/provider.

For user display, current culture may be appropriate.

Performance Considerations

String performance becomes important primarily when code:

```
EXAMPLE / FLOW
Processes large files

Builds huge outputs

Runs millions of string operations

Parses high-volume network protocols

Creates many temporary substrings

Runs in hot application paths
```

For ordinary business logic, choose clarity first.

Allocation Consideration

STRINGS / C#

Because strings are immutable:

```
C#
value =
    value.Trim()
         .Replace(...)
         .ToUpperInvariant();
```

can produce multiple intermediate strings.

Usually fine.

If this occurs millions of times:

```
EXAMPLE / FLOW
measure
```

before optimizing.

StringBuilder

Useful when:

```
EXAMPLE / FLOW
Many incremental modifications
```

especially large loops.

But not automatically faster for every small string expression.

Span

Useful when:

```
EXAMPLE / FLOW
Substring allocations
```

are a measured bottleneck in high-volume parsing.

Microsoft's current string-operation guidance says normal string methods should be preferred for simple searches, while spans are useful for hot paths and large inputs where allocations matter.

String Comparison Performance

For non-linguistic identifiers:

```
EXAMPLE / FLOW
Ordinal
OrdinalIgnoreCase
```

are not only semantically appropriate but generally faster than culture-aware linguistic comparisons.

Correctness should determine the comparison mode first.

String Interning

STRINGS / C#

Manual interning can reduce duplicate storage in narrow scenarios, but it can also keep strings alive for a long time.

Do not use it as a default optimization.

Security Considerations

Strings frequently cross trust boundaries:

EXAMPLE / FLOW

```
HTTP request

User input

Database

Files

External APIs

Headers

URLs
```

Never assume incoming text is safe.

SQL Injection

Use parameterized SQL.

Never build SQL by concatenating untrusted strings.

HTML / JavaScript Output

Do not manually concatenate user input into HTML or JavaScript without the appropriate encoding framework.

Use framework templating/encoding systems.

URL Handling

Encode user-controlled query values correctly.

Do not simply concatenate arbitrary raw user input into URLs.

File Paths

Do not trust user-provided path text without validation.

Potential issues include:

EXAMPLE / FLOW

```
Path traversal

Unexpected root paths
```

```
Illegal characters
*
Access outside allowed directories
```

STRINGS / C#

Use path APIs such as:

```C#
Path.Combine(...)
```

rather than manually adding slash characters.

Logging

Avoid logging secrets such as:

EXAMPLE / FLOW

```
Passwords

JWTs

API keys

Card data

Private tokens
```

Strings make it easy to accidentally persist sensitive information into logs.

Case-Insensitive Security Comparison

Never invent security-sensitive comparison rules using:

```C#
ToLower()
```

Use the exact comparison defined by the protocol or specification.

For culture-independent security/technical identifiers, explicit ordinal comparison is generally appropriate.

Practical Exercise — Reverse a String

A common interview problem:

```C#
static string Reverse(
    string input)
{
    var builder =
        new StringBuilder(
            input.Length);

    for (int i =
             input.Length - 1;
         i >= 0;
         i--)
    {
        builder.Append(
```

```
            input[i]);
    }

    return builder.ToString();
}
```

STRINGS / C#

For basic ASCII interview strings this demonstrates:

EXAMPLE / FLOW

```
Indexing

Looping

StringBuilder
```

For full Unicode grapheme correctness, reversing `char` values is not enough.

Practical Exercise — Palindrome

C#

```
static bool IsPalindrome(
    string input)
{
    int left =
        0;

    int right =
        input.Length - 1;

    while (left < right)
    {
        if (input[left]
            !=
            input[right])
        {
            return false;
        }

        left++;
        right--;
    }

    return true;
}
```

For:

EXAMPLE / FLOW

```
LEVEL
```

result:

EXAMPLE / FLOW

```
true
```

Again, this is a basic `char`-oriented interview solution.

Practical Exercise — Count Character Frequency

STRINGS / C#

C#

```
static Dictionary<char, int>
    CountCharacters(
        string input)
{
    var counts =
        new Dictionary<char, int>();

    foreach (char character
             in input)
    {
        if (counts.TryGetValue(
                character,
                out int count))
        {
            counts[character] =
                count + 1;
        }
        else
        {
            counts[character] =
                1;
        }
    }

    return counts;
}
```

Later Collections will explain the dictionary in detail.

Practical Exercise — First Non-Repeating Character

C#

```
static char? FindFirstNonRepeating(
    string input)
{
    var counts =
        new Dictionary<char, int>();

    foreach (char character
             in input)
    {
        counts[character] =
            counts.TryGetValue(
                character,
                out int count)
                ? count + 1
                : 1;
    }

    foreach (char character
             in input)
    {
        if (counts[character] == 1)
        {
            return character;
        }
```

```
    }

    return null;
}
```

STRINGS / C#

This is a classic interview problem.

Practical Exercise — Normalize Currency

Input:

EXAMPLE / FLOW

```
" usd "
```

Solution:

C#

```
static string NormalizeCurrencyCode(
    string value)
{
    ArgumentException.ThrowIfNullOrWhiteSpace(
        value);

    return value
        .Trim()
        .ToUpperInvariant();
}
```

Result:

EXAMPLE / FLOW

```
USD
```

Practical Exercise — Parse Gift Reference

C#

```
static bool TryParseGiftReference(
    string reference,
    out int year,
    out long giftNumber)
{
    year =
        default;

    giftNumber =
        default;

    string[] parts =
        reference.Split(
            '-',
            StringSplitOptions.TrimEntries);

    if (parts.Length != 3)
    {
        return false;
    }
```

```
    if (!string.Equals(
            parts[0],
            "GFT",
            StringComparison.Ordinal))
    {
        return false;
    }

    return int.TryParse(
                parts[1],
                out year)
            &&
            long.TryParse(
                parts[2],
                out giftNumber);
}
```

STRINGS / C#

Input:

EXAMPLE / FLOW

```
GFT-2026-10015
```

Output:

EXAMPLE / FLOW

```
year = 2026

giftNumber = 10015
```

Interview Questions

What is a string in C#?

A string is an instance of `System.String` representing text as a sequence of UTF-16 `char` code units.

`string` is the C# alias for `System.String`.

Is string a value type or reference type?

`string` is a reference type.

However, it is immutable and defines value-oriented equality behavior, which can make it feel different from ordinary mutable reference types.

What does string immutability mean?

A string object's contents cannot change after creation.

Operations such as:

C#

```
Replace
Trim
ToUpper
+
```

produce new strings rather than modifying the original string.

Why are strings immutable?

Immutability supports safe sharing, stable hashing, thread-friendly reads, predictable behavior, and runtime optimizations such as interning.

What happens when you write s += "x"?

A new resulting string is created and assigned back to `s`.

The original string object is unchanged.

What is the difference between null, empty, and whitespace?

`null`:

EXAMPLE / FLOW

```
No string object/value.
```

Empty:

EXAMPLE / FLOW

```
""
Length = 0
```

Whitespace:

EXAMPLE / FLOW

```
Contains whitespace characters
but no meaningful visible text.
```

What is IsNullOrEmpty?

It returns true when a string is:

EXAMPLE / FLOW

```
null
or
empty
```

What is IsNullOrWhiteSpace?

It returns true when the input is:

EXAMPLE / FLOW

```
null

empty

or consists only of whitespace
```

This is often appropriate for user-input validation.

What is the difference between `string` and `String`?

There is no runtime type difference.

```C#
string
```

is the C# alias for:

```C#
System.String
```

What is a verbatim string?

A string prefixed with:

```EXAMPLE / FLOW
@
```

that reduces escaping of backslashes and supports literal line breaks.

Example:

```C#
@"C:\Reports\Gifts"
```

What is a raw string literal?

A modern C# literal surrounded by at least three double quotes:

```C#
"""
text
"""
```

It allows multi-line text and embedded characters with much less escaping.

When are raw strings useful?

Particularly for:

```EXAMPLE / FLOW
JSON

XML

SQL

HTML

Regex

Large templates
```

What is string interpolation?

Interpolation inserts expressions into a string using:

C#

```
$"...{expression}..."
```

STRINGS / C#

Example:

C#

```
$"Gift {giftId}"
```

What is StringBuilder?

`StringBuilder` is a mutable text buffer used when many string modifications/appends are needed.

It avoids creating a new immutable string for every intermediate modification.

When should you use StringBuilder?

Use it when constructing text incrementally many times, especially in large loops or large text-generation operations.

Do not automatically use it for a few small concatenations.

What is the difference between string and StringBuilder?

`string`:

EXAMPLE / FLOW

```
Immutable
```

`StringBuilder`:

EXAMPLE / FLOW

```
Mutable text buffer
```

StringBuilder eventually produces a `string` using:

C#

```
ToString()
```

How does == work for strings?

`string` overloads `==` to compare string values rather than ordinary object reference identity.

For more explicit comparison semantics, use `string.Equals` with `StringComparison`.

Why should StringComparison be specified?

Because string comparison can be:

EXAMPLE / FLOW

```
Ordinal

Culture-sensitive

Case-sensitive

Case-insensitive
```

The correct option depends on the meaning of the text.

Explicit comparison prevents ambiguity and subtle globalization/security problems.

When should you use Ordinal?

For culture-independent, non-linguistic values such as many:

EXAMPLE / FLOW

```
Technical identifiers

Protocol tokens

Internal codes

Security identifiers
```

When should you use OrdinalIgnoreCase?

For culture-independent identifiers where casing should not matter.

Example:

EXAMPLE / FLOW

```
USD
usd
```

when your application's currency-code rules are case-insensitive.

Why not compare using ToLower()?

It:

EXAMPLE / FLOW

```
may allocate a new string

may involve culture rules

hides comparison intent
```

Use an explicit case-insensitive comparer instead.

What does Length represent?

The number of UTF-16 `char` code units in the string.

It does not necessarily equal the number of Unicode scalar values or human-visible characters.

Is char always one visible character?

No.

Some Unicode scalar values require two UTF-16 `char` values, and some displayed grapheme clusters can contain several Unicode scalar values.

What is Rune?

`System.Text.Rune` represents a Unicode scalar value.

It is useful when algorithms need Unicode code-point-aware processing rather than raw UTF-16 `char` processing.

What is string interning?

Interning stores shared references for equal string values in an intern pool.

`String.Intern` can retrieve/add an interned representation.

Should application developers manually intern normal strings?

Usually no.

Interned strings can remain alive for long periods, so indiscriminate interning can increase memory retention.

What is ReadOnlySpan used for?

It provides a lightweight read-only view over character memory.

It can help high-performance parsing avoid creating new substrings for every slice.

Should Span replace normal string methods?

No.

Use normal string methods for ordinary application code.

Use span-based processing when profiling or workload characteristics show that substring/split allocations matter.

What does Split do?

It divides one string into multiple substrings according to delimiters and options.

What does Join do?

It combines multiple strings using a separator.

Example:

```C#
string.Join(
    ",",
    values);
```

What does Trim do?

It removes whitespace or specified boundary characters from the beginning and end of a string.

It does not remove interior whitespace.

What is the difference between Replace and modifying a string?

`Replace` returns a new string.

The original remains unchanged because strings are immutable.

Why should decimal values remain decimal until display?

Because:

EXAMPLE / FLOW

```
decimal
```

STRINGS / C#

preserves numeric semantics and supports arithmetic.

Convert to formatted strings only when presenting or transmitting the value.

Why is manual JSON construction discouraged?

Because proper serialization must handle:

EXAMPLE / FLOW

```
escaping

quotes

Unicode

null values

nested objects
```

Use a JSON serializer.

Why is SQL string concatenation dangerous?

Concatenating untrusted input into SQL can create SQL injection vulnerabilities.

Use parameterized SQL.

What is a UTF-8 string literal?

A literal with:

EXAMPLE / FLOW

```
u8
```

such as:

C#

```
"AUTH "u8
```

which is represented as `ReadOnlySpan<byte>` containing UTF-8 data rather than a normal UTF-16 `string`.

Best Practices

Use:

C#

```
string
```

for textual domain values.

Remember that strings are immutable.

Always store the result of modifying methods such as:

EXAMPLE / FLOW

```
Trim

Replace

ToUpper

ToLower
```

STRINGS / C#

unless you intentionally ignore it.

Use:

C#

```
string.IsNullOrWhiteSpace(...)
```

for many user-required text validations.

Keep:

EXAMPLE / FLOW

```
null
empty
whitespace
```

distinct when business meaning differs.

Use interpolation for readable normal message construction.

Use raw strings for complex multi-line literal content.

Use `StringBuilder` for substantial incremental text construction.

Use `string.Join` when combining collections with delimiters.

Use explicit:

C#

```
StringComparison
```

for comparisons.

Use:

EXAMPLE / FLOW

```
Ordinal / OrdinalIgnoreCase
```

for non-linguistic technical identifiers unless a specification requires something else.

Use culture-aware comparison and formatting for actual human linguistic text.

Do not use `ToLower()` simply to implement case-insensitive comparison.

Keep numeric/date values strongly typed until presentation or serialization.

Use proper serializers for JSON/XML.

Use parameterized queries for SQL.

Use path APIs for file paths.

Be Unicode-aware when processing arbitrary international text.

Do not assume:

EXAMPLE / FLOW

```
Length == visible characters
```

for Unicode data.

Use `Span` / `Rune` only when the problem genuinely requires their capabilities.

Do not prematurely optimize ordinary string code.

Measure allocation/performance issues before introducing low-level complexity.

Quick Revision

Concept	Meaning
`string`	Alias for `System.String`
String type	Reference type
Immutability	String object cannot change after creation
`Length`	Number of UTF-16 `char` values
`"..."`	Regular string literal
`@"..."`	Verbatim string
`"""..."""`	Raw string literal
`$"..."`	Interpolated string
`$"""..."""`	Interpolated raw string
`+`	String concatenation
`StringBuilder`	Mutable text construction
`Concat`	Combine values
`Join`	Combine sequence with separator
`Split`	Break string into parts
`Contains`	Search for substring
`StartsWith`	Test prefix
`EndsWith`	Test suffix
`IndexOf`	Find position
`Substring`	Create substring
`Replace`	Return text with replacements
`Trim`	Remove boundary whitespace
`IsNullOrEmpty`	Null or zero-length
`IsNullOrWhiteSpace`	Null/empty/whitespace
`Ordinal`	Culture-independent comparison
`OrdinalIgnoreCase`	Case-insensitive ordinal comparison
`Rune`	Unicode scalar value
UTF-16	Encoding model used by .NET strings
`"text"u8`	UTF-8 byte literal
`ReadOnlySpan<char>`	Allocation-free character view
Interning	Shared string reference pool

Complete Fundraising Example

C#

```
using System.Globalization;
using System.Text;

public sealed class Gift
{
    public long GiftId
    {
        get;
        init;
    }

    public long ConstituentId
    {
        get;
        init;
    }

    public decimal TotalAmount
    {
        get;
        init;
    }

    public DateOnly GiftDate
    {
        get;
        init;
    }

    public string CurrencyCode
    {
        get;
        init;
    } = "USD";

    public string? ReferenceNumber
    {
        get;
        init;
    }

    public string? Notes
    {
        get;
        init;
    }
}

static string NormalizeCurrencyCode(
    string currencyCode)
{
    ArgumentException.ThrowIfNullOrWhiteSpace(
        currencyCode);

    return currencyCode
        .Trim()
        .ToUpperInvariant();
}
```

```
static bool IsUsdGift(
    Gift gift)
{
    return string.Equals(
        gift.CurrencyCode,
        "USD",
        StringComparison.OrdinalIgnoreCase);
}

static string GetNotesDisplay(
    Gift gift)
{
    return string.IsNullOrWhiteSpace(
               gift.Notes)

        ? "No notes"

        : gift.Notes.Trim();
}

static string BuildReceipt(
    Gift gift,
    string donorName)
{
    string campaignReference =
        gift.ReferenceNumber
        ??
        "No Reference";

    return $"""
    FUNDRAISING PLATFORM
    GIFT RECEIPT

    Gift ID: {gift.GiftId}
    Donor: {donorName}
    Date: {gift.GiftDate:yyyy-MM-dd}
    Amount: {gift.TotalAmount:C2}
    Currency: {gift.CurrencyCode}
    Reference: {campaignReference}
    Notes: {GetNotesDisplay(gift)}
    """;
}

var gift =
    new Gift
    {
        GiftId =
            10001,

        ConstituentId =
            5001,

        TotalAmount =
            1250.50m,

        GiftDate =
            new DateOnly(
                2026,
                10,
                29),
```

```
        CurrencyCode =
            NormalizeCurrencyCode(
                " usd "),

        ReferenceNumber =
            "GFT-2026-10001",

        Notes =
            "  Annual appeal gift  "
    };

string receipt =
    BuildReceipt(
        gift,
        "John Smith");

Console.WriteLine(
    receipt);
```

STRINGS / C#

The example combines:

```
String validation

Trim

Case normalization

OrdinalIgnoreCase comparison

Null handling

Raw strings

Interpolation

Currency formatting

Date formatting
```

Complete String Processing Flow

```
Text Enters Application
    →
Where did it come from?
```

User input:

```
Validate
    →
Null / Empty / Whitespace?
    →
Trim if appropriate
    →
Validate business rules
```

Technical identifier:

EXAMPLE / FLOW

```
Compare
    →
Ordinal
or
OrdinalIgnoreCase
```

Human linguistic text:

EXAMPLE / FLOW

```
Culture-aware behavior
when appropriate
```

Transform:

EXAMPLE / FLOW

```
Replace
Trim
Substring
Split
Join
```

Build output:

EXAMPLE / FLOW

```
Small text
    →
Interpolation / Concat

Large repeated construction
    →
StringBuilder
```

Machine conversion:

EXAMPLE / FLOW

```
String
    →
Parse
    →
decimal / long / DateTime / etc.
```

Display conversion:

EXAMPLE / FLOW

```
Strongly typed value
    →
Formatting
    →
User-facing string
```

High-performance parsing:

EXAMPLE / FLOW

```
Large / Hot Path
    →
ReadOnlySpan<char>
    →
Slice without unnecessary strings
```

Unicode-sensitive processing:

EXAMPLE / FLOW

```
Need scalar-value correctness?
    →
Rune / Unicode-aware APIs
```

Web transport:

EXAMPLE / FLOW

```
C# string
    →
UTF-16 internally
    →
Serializer / Encoder
    →
UTF-8 network bytes
```

Final Concept Flow

EXAMPLE / FLOW

```
string
    →
Reference Type
    →
Immutable
```

Therefore:

EXAMPLE / FLOW

```
"Gift"
    →
Replace / Trim / Concatenate
    →
New String
```

Comparison:

EXAMPLE / FLOW

```
Business meaning
    →
Human text?
        →
Culture-sensitive when appropriate

Technical text?
        →
Ordinal / OrdinalIgnoreCase
```

Construction:

EXAMPLE / FLOW

```
Few parts
    →
Interpolation

Many repeated parts
    →
StringBuilder
```

Processing:

EXAMPLE / FLOW

```
Small / normal workload
    →
String methods

Large performance-sensitive workload
    →
Span-based processing when justified
```

Unicode:

EXAMPLE / FLOW

```
string
    →
UTF-16 char units
    →
One displayed character
may use multiple char values
```

The central lesson is:

> **A C# string is an immutable reference type. Professional string handling requires more than concatenation: you must understand null/empty states, comparison semantics, culture, formatting, Unicode, allocation behavior, and safe handling of text that crosses API, database, file, and user-input boundaries.**

For a full-stack .NET developer, these concepts directly affect:

EXAMPLE / FLOW

```
ASP.NET Core APIs

DTO validation

Logging

JSON

SQL

URLs

Localization

Authentication tokens
```

```
Search

Data imports

Performance

Security
```

Next roadmap topic: Arrays, Indexes, and Ranges.